

Respostas - Evolução das Principais Linguagens de Programação

Exercícios selecionados: 1, 4, 7, 10, 12, 14 e 20

1. A genealogia das linguagens não é uma escada de progresso.

A evolução das linguagens de programação não acontece de forma linear, em que uma linguagem simplesmente substitui a anterior. Diferentes linguagens surgiram para atender a necessidades, domínios e públicos diferentes.

Dois fatores históricos que permitem uma linguagem influenciar outra sem substituí-la são:

- Necessidades específicas: uma linguagem pode ser criada para resolver problemas de determinado domínio. Por exemplo, Fortran foi direcionada ao cálculo científico, enquanto COBOL foi voltada ao processamento comercial.
- Influência de conceitos: uma linguagem pode introduzir ideias que são incorporadas por linguagens posteriores, mesmo que ela própria deixe de ser utilizada. Assim, seus conceitos permanecem na evolução das linguagens.

4. Por que o projeto Fortran precisou convencer programadores de que código traduzido podia competir com código de máquina?

Quando Fortran surgiu, muitos programadores acreditavam que programas escritos em linguagem de alto nível seriam mais lentos do que programas escritos diretamente em linguagem de máquina.

Por isso, o projeto precisava demonstrar que o compilador poderia produzir código suficientemente eficiente. Se o desempenho fosse próximo ao obtido com código escrito manualmente, os programadores poderiam economizar muito tempo e custo de desenvolvimento, sem perder desempenho significativo.

Dessa forma, o sucesso do Fortran dependia de equilibrar desempenho e produtividade, facilitando sua adoção pelos programadores.

7. COBOL foi desenhada para processamento comercial.

O COBOL foi desenvolvido pensando principalmente em processamento de informações comerciais, como registros, arquivos e grandes volumes de dados. Por isso, sua linguagem buscava ser mais próxima da linguagem utilizada no ambiente empresarial, favorecendo a legibilidade.

Seus recursos para representar registros também eram importantes para organizar informações comerciais de maneira estruturada.

Além disso, o COBOL teve relação com o FLOW-MATIC, linguagem desenvolvida anteriormente para processamento de dados empresariais. Vários conceitos e ideias do FLOW-MATIC influenciaram o desenvolvimento do COBOL.

10. Ortogonalidade e ALGOL 68

Ortogonalidade é uma característica de projeto em que os recursos de uma linguagem podem ser combinados de maneira consistente, sem criar muitas exceções ou regras especiais.

A ALGOL 68 buscou um alto grau de ortogonalidade e regularidade. Isso significa que seus elementos podiam ser utilizados de forma mais uniforme em diferentes situações.

Porém, regularidade não significa necessariamente simplicidade. Uma linguagem pode possuir regras muito consistentes, mas apresentar muitos recursos e combinações possíveis, tornando seu aprendizado difícil.

Portanto, uma linguagem muito ortogonal não é automaticamente fácil de usar. A ortogonalidade pode aumentar a consistência, mas a quantidade e a complexidade dos recursos também influenciam a facilidade de utilização.

12. Pequena base Prolog

Podemos representar uma pequena base de conhecimento em linguagem natural:

- Fato 1: João é estudante.
- Fato 2: Todo estudante estuda.
- Regra: Se uma pessoa é estudante, então ela estuda.
- Consulta: João estuda?

Em Prolog, a ideia poderia ser representada conceitualmente como:

```
estudante(joao).  
estuda(X) :- estudante(X).
```

A consulta seria:

```
estuda(joao).
```

O resultado seria verdadeiro, pois existe um fato informando que João é estudante e uma regra que permite concluir que estudantes estudam.

Isso representa programação lógica porque o programa descreve fatos e relações, e o mecanismo da linguagem utiliza regras para inferir respostas às consultas. Não se trata apenas de armazenar informações.

14. Objetos em Smalltalk, C++ e Java

Em Smalltalk, o conceito de objetos é central para a linguagem. O sistema é fortemente baseado na orientação a objetos, com objetos interagindo por meio de mensagens.

O C++ incorporou recursos de orientação a objetos mantendo uma forte relação com o C. Isso permitiu que programadores utilizassem conceitos como classes e objetos sem abandonar completamente características e compatibilidade relacionadas à linguagem C.

O Java também utiliza orientação a objetos, mas adotou uma estratégia diferente para obter portabilidade. Seus programas são compilados para bytecode, que pode ser executado pela Máquina Virtual Java (JVM) em diferentes sistemas.

Assim:

- Smalltalk: orientação a objetos como fundamento central.
- C++: orientação a objetos integrada à base existente do C.
- Java: orientação a objetos combinada com uma estratégia de portabilidade baseada na JVM.

20. Estudo de caso

Para uma equipe que precisa trabalhar em diferentes áreas, eu escolheria famílias de linguagens de acordo com o domínio:

- Cálculo científico — Fortran: foi desenvolvida com forte foco em computação científica e numérica.
- Regras declarativas — Prolog: é baseada no paradigma de programação lógica e permite representar fatos, regras e consultas.
- Aplicação Web interativa — JavaScript: foi criada para permitir programação em páginas Web e tornou-se uma linguagem central para aplicações interativas.
- Firmware restrito — C: possui características adequadas à programação de baixo nível e ao controle eficiente de recursos.

Dois trade-offs importantes seriam:

1. Desempenho × facilidade de desenvolvimento: linguagens de baixo nível, como C, oferecem maior controle sobre os recursos, mas normalmente exigem mais cuidado do programador.
2. Abstração × controle: linguagens de alto nível facilitam o desenvolvimento por oferecerem maior abstração, mas podem reduzir o controle direto sobre hardware e recursos do sistema.

Essas escolhas seguem a evolução histórica apresentada no exercício, que relaciona diferentes linguagens aos domínios e problemas que motivaram seu desenvolvimento.